

06048617

1. Exploratory Data Analysis

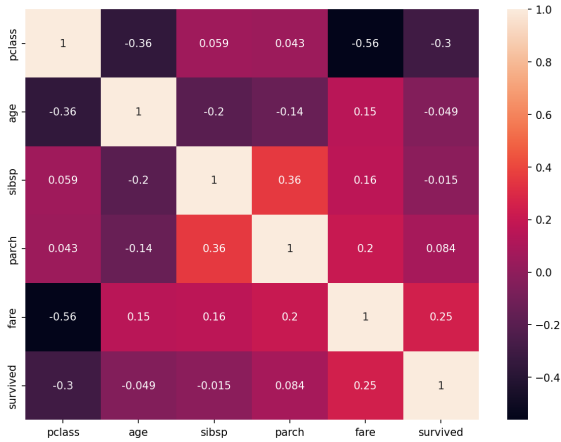


Figure 1. Pearson correlation heatmap for numerical features in the Titanic dataset.

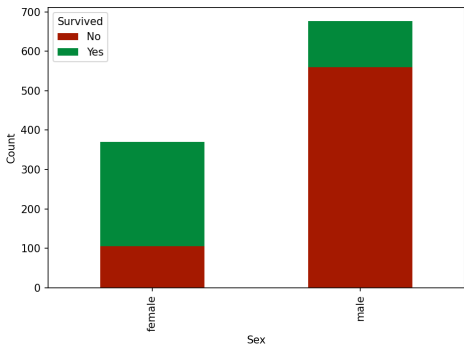


Figure 2. Survival distribution by sex, showing the “women and children first” pattern.

1.1. Trends and Patterns in the Data

The correlation heatmap (Figure 1) reveals several notable patterns. Passenger class (`pclass`) shows a negative correlation with survival ($r \approx -0.34$), indicating that first-class passengers (`pclass=1`) had higher survival rates than third-class passengers. This aligns with the positive correlation between `fare` and survival ($r \approx 0.26$), as wealthier passengers could afford better accommodations closer to lifeboats. The strong negative correlation between `pclass` and `fare` ($r \approx -0.55$) confirms this relationship.

The sex distribution plot (Figure 2) reveals a stark disparity: the majority of female passengers survived, while most male passengers perished. This reflects the historical “women and children first” evacuation protocol. Family-related features (`sibsp` and `parch`) show weak correla-

tions with survival, suggesting family size had limited direct impact on survival outcomes.

1.2. Expected Feature Importance

Based on the EDA, `sex` is expected to be the most important feature due to the pronounced survival difference between males and females. `Pclass` and `fare` should also rank highly given their correlations with survival. Conversely, `sibsp` and `parch` are expected to be least important due to their weak correlations with the target variable.

However, this assessment carries uncertainty. Pearson correlation only captures linear relationships, whereas neural networks can learn complex non-linear patterns. Features with low linear correlation may still contribute through interactions with other features. Additionally, the preprocessing (scaling, one-hot encoding) may affect how the model weighs different features.

1.3. Additional Exploratory Analysis

Several additional analyses could enhance understanding: (1) examining feature distributions through histograms to identify skewness and outliers; (2) creating class-conditional distributions (e.g., age distribution for survivors vs. non-survivors); (3) analyzing feature interactions through pairwise scatter plots or grouped statistics (e.g., survival rate by age group and sex); (4) investigating missing value patterns, as missingness itself may be informative; and (5) computing mutual information scores to capture non-linear dependencies between features and the target variable.

2. Feature Attribution Explanations

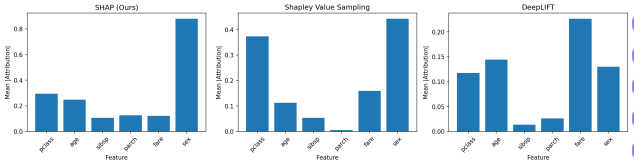


Figure 3. Mean absolute feature attributions for SHAP, SVS, and DeepLIFT across 10 test samples.

2.1. Attribution Method Comparison

We computed feature attributions using three methods (Figure 3): exact SHAP (our implementation), Shapley Value Sampling (SVS), and DeepLIFT on 10 randomly selected test samples. Table 1 shows mean absolute attribution scores by feature.

Most important features: SHAP and SVS rank `sex` as most important (0.88 and 0.44), followed by `pclass`.

Feature	SHAP	SVS	DeepLIFT
pclass	0.295	0.374	0.117
age	0.249	0.112	0.144
sibsp	0.105	0.054	0.013
parch	0.126	0.005	0.026
fare	0.122	0.160	0.226
sex	0.880	0.443	0.130

Table 1. Mean absolute feature attributions across 10 test samples.

However, DeepLIFT ranks `fare` highest (0.23), with `sex` ranking only third. This aligns with our EDA expectations for SHAP/SVS—the model learned the historical “women and children first” protocol.

Least important features: `sibsp` and `parch` showed the lowest attribution magnitudes across all methods, confirming that family size has minimal predictive impact.

Differences between methods: SHAP and SVS agree on the ranking (`sex` > `pclass` > `fare`), but DeepLIFT produces a strikingly different ordering (`fare` > `age` > `sex`). This occurs because DeepLIFT uses gradient-based back-propagation measuring local sensitivity, whereas SHAP computes average marginal contributions across all feature coalitions. DeepLIFT may overweight features with large gradient magnitudes (like `fare`) while underweighting categorical features like `sex` whose one-hot encoding dilutes gradients.

Comparison to EDA expectations: SHAP and SVS matched our predictions well, with `sex` dominant. DeepLIFT’s divergent ranking illustrates a key limitation: gradient-based methods may not capture the true feature importance for models with categorical inputs or non-smooth decision boundaries.

Advantages/Disadvantages: SHAP provides theoretically grounded, consistent attributions but is computationally expensive ($O(2^n)$). SVS offers a practical approximation with controllable accuracy-speed tradeoffs. DeepLIFT is fastest but model-specific and may miss feature interactions captured by perturbation methods.

2.2. Infidelity Evaluation

Table 2 shows infidelity scores (lower is better) across different perturbation settings.

Surprisingly, all three methods achieved similar infidelity scores, with DeepLIFT slightly outperforming SHAP and SVS (mean 0.131 vs 0.133). This contrasts with expectations that perturbation-based methods would better align with the perturbation-based infidelity metric. The similar performance suggests all methods adequately capture the model’s local behaviour. Infidelity increases with noise scale (σ), as larger perturbations push inputs further from the training distribution where linear approximations break down.

σ	p_{cat}	SHAP	SVS	DeepLIFT
0.1	0.1	0.075	0.074	0.073
0.1	0.3	0.079	0.078	0.077
0.1	0.5	0.080	0.080	0.082
0.2	0.1	0.122	0.122	0.121
0.2	0.3	0.129	0.128	0.126
0.2	0.5	0.129	0.130	0.128
0.5	0.1	0.186	0.190	0.187
0.5	0.3	0.193	0.196	0.192
0.5	0.5	0.202	0.203	0.197
Mean		0.133	0.133	0.131

Table 2. Infidelity scores for different noise scales (σ) and categorical resampling probabilities (p_{cat}). Lower values indicate better faithfulness.

2.3. Computational Efficiency

Table 3 compares runtime for computing attributions on 200 test samples.

Dataset	Features	SHAP	SVS	DeepLIFT
Titanic	6	18.7s	0.06s	0.01s
Dry Bean	16	56,675s*	0.12s	0.01s

Table 3. Runtime (seconds) for 200 samples. *Dry Bean SHAP extrapolated from 1 sample due to computational constraints.

DeepLIFT is orders of magnitude faster due to its single backward pass ($O(1)$ per sample). SVS offers a middle ground with sampling-based approximation. Exact SHAP becomes impractical for datasets with many features—Dry Bean (16 features) requires $2^{16} = 65,536$ coalition evaluations per sample versus Titanic’s $2^6 = 64$. The extrapolated SHAP time for Dry Bean (~ 15.7 hours for 200 samples) demonstrates the exponential scaling problem. The Dry Bean model achieved 92.9% accuracy.

3. Counterfactual Explanations

3.1. Distance Metric Design

Standard L1 on preprocessed data: Using unweighted L1 distance $d(x, x') = \sum_i |x_i - x'_i|$ on the preprocessed Titanic data creates unequal feature weighting. The one-hot encoded `sex` feature occupies two columns (`sex_female`, `sex_male`), effectively doubling its weight compared to single-column features. Additionally, the RobustScaler transforms continuous features to different ranges depending on their original distributions, causing features with larger scaled ranges to dominate the distance.

Normalized L1 on preprocessed data: Normalizing by feature range $d(x, x') = \sum_i |x_i - x'_i| / (\max_i - \min_i)$ partially addresses scale differences but still gives `sex` double weight due to its two-column encoding. Furthermore, normalization ranges must be computed from training data, and outliers can skew min/max values.

Our design for equal original feature weighting: To

treat each of the 6 original features equally regardless of preprocessing:

$$d(x_1, x_2) = \frac{1}{n} \sum_{i=1}^n \frac{|x_1^{(i)} - x_2^{(i)}|}{r_i} \quad (1)$$

where $n = 6$ is the number of original features and r_i is the normalization range. For continuous features (pclass, age, sibsp, parch, fare), we use $r_i = 4$ to account for the RobustScaler’s typical output range (roughly $[-2, 2]$ for most data). For the one-hot encoded sex feature, we sum differences across both columns and normalize by 2 (maximum difference when switching $[1, 0] \leftrightarrow [0, 1]$), then contribute 1/6 to the total—the same weight as each continuous feature.

This ensures: (1) equal 1/6 weight per original feature regardless of encoding, (2) scale-invariance through normalization, and (3) interpretable distances where $d \in [0, 1]$.

3.2. NNCE vs WAC Comparison

Table 4 compares Nearest Neighbour Counterfactual Explanations (NNCE) and Wachter’s Algorithm for Counterfactuals (WAC) across three metrics evaluated on 100 test samples.

Method	Validity	Proximity (Cost)	Plausibility
NNCE	1.00 ± 0.00	0.023 ± 0.004	0.006 ± 0.002
WAC	0.91 ± 0.06	0.021 ± 0.002	0.026 ± 0.003

Table 4. Counterfactual evaluation metrics (mean $\pm$ std over 5 runs of 20 samples). Validity: fraction achieving class flip. Proximity: distance to original. Plausibility: average distance to 5 nearest training neighbours.

Validity: NNCE achieves perfect validity (1.0) because it selects actual training points with the opposite predicted label by construction. WAC achieves 91% validity, with some counterfactuals failing to cross the decision boundary despite optimization, likely due to local minima or insufficient iterations.

Proximity: WAC achieves slightly lower cost (0.021 vs 0.023), as gradient descent can find points closer to the decision boundary than the nearest training example. However, the difference is small, suggesting NNCE finds reasonably proximal counterfactuals.

Plausibility: NNCE dramatically outperforms WAC (0.006 vs 0.026). Plausibility measures average distance to 5 nearest training neighbours—lower indicates the counterfactual lies in a high-density region of the data manifold. Since NNCE returns actual training points, they are inherently plausible. WAC counterfactuals may lie in sparse regions of feature space, producing implausible explanations that do not resemble real data.

3.3. Discussion

The results highlight a fundamental trade-off in counterfactual generation. NNCE guarantees validity and plausibility by restricting counterfactuals to observed data, but may sacrifice proximity when no nearby opposite-class examples exist. WAC can find minimal perturbations but risks generating implausible or invalid counterfactuals.

For applications requiring trustworthy explanations (e.g., loan decisions), NNCE’s guaranteed validity and plausibility are advantageous—users receive actionable explanations based on real cases. For exploratory analysis where proximity is paramount, WAC may reveal the minimal changes needed to alter predictions.

The choice depends on the use case: NNCE for high-stakes decisions requiring realistic alternatives; WAC for understanding model sensitivity and decision boundaries.